

mf_algebra documentation

a manim plugin for beautiful, meaningful, and automatic algebra animations.
compatible with both manimgl and manim community edition.

author: john connell (the mathematic fanatic)

table of contents

1.	installation	2
2.	overview	3
3.	the three pillars	3
4.	expressions	3
4.1.	building expressions	3
4.2.	variables	4
4.3.	numbers	4
4.4.	operations	5
4.4.1.	nesting and de-nesting	5
4.4.2.	computing values	5
4.5.	relations and equations	6
4.6.	functions	6
4.6.1.	parenthesization modes	7
4.6.2.	built-in function types	7
4.7.	scripts (subscripts and superscripts)	8
4.8.	sequences and coordinates	8
4.9.	parentheses	8
5.	the address system	8
5.1.	what addresses are	9
5.2.	special address characters	9
5.3.	glyphs and the mob	10
5.4.	querying addresses	10
6.	actions	10
6.1.	built-in actions	11
6.1.1.	substitution	11
6.1.2.	function expansions	11
6.1.3.	applying operations to both sides	11
6.1.4.	evaluation	12
6.1.5.	permutations and swapping	12
6.1.6.	parallel actions	12
6.1.7.	algebraic actions and template matching	12
6.1.8.	simplification rules	13
6.1.9.	equation maneuvers	14

6.1.10.	wrapping and reciprocals	15
6.1.11.	distribution and factoring	15
6.1.12.	action modifiers	15
6.1.13.	address maps and animations	15
6.1.14.	custom actions	16
7.	timelines	17
7.1.	building timelines with <code>>></code>	17
7.2.	manual timelines	17
7.3.	auto timelines	18
7.4.	evaluate timeline	18
7.5.	solve timeline	18
7.6.	playing animations	19
7.7.	timeline options	19
8.	calculus	20
8.1.	differentials	20
8.2.	differentiation	20
8.3.	integrals	21
8.4.	limits	21
8.5.	series	22
9.	trigonometry	22
10.	sets	23
11.	logic	23
12.	utilities	24
12.1.	configuration	24
12.2.	smarten	24
12.3.	conversions	25
12.4.	serialization	25
12.5.	random expression generation	25
13.	operator overload reference	26
13.1.	expression operators	26
13.2.	action operators	26
13.3.	the <code>>></code> pipeline	26
14.	project structure	27

1. installation

```
pip install MF_Algebra
```

use with manimgl:

```
from manimlib import *
from MF_Algebra import *
```

or with manim community edition:

```
from manim import *
from MF_Algebra import *
```

dependencies: `MF_Tools ≥ 1.4.3`, `sympy`, `numpy ≥ 2.0`, `tabulate`, `dill`, `asteval`.
one of `manimgl ≥ 1.7.0` or `manim ≥ 0.17.0` is required.

2. overview

`mf_algebra` lets you build math expressions as tree structures, transform them through algebraic actions, and animate the whole process in manim. you work with three kinds of objects, and they all compose together with the `>>` operator.

3. the three pillars

concept	what it is	example
expression	a tree representing a math formula	<code>x**2 + 3*x + 1</code>
action	a transformation between expressions	<code>substitute_({x: 5})</code>
timeline	a sequence of expressions and actions	<code>(x**2 + 1) >> substitute_({x: 3})</code> <code>>> evaluate_()</code>

every expression produces a manim `Tex` mobject. every action knows how to produce an output expression from an input expression, and how to animate the transition. timelines manage the sequence of steps and drive playback in a scene.

4. expressions

4.1. building expressions

expressions are built using python operators. the library pre-defines single-letter variable objects so you can write math naturally:

```
from MF_Algebra import *
```

```

expr = x**2 + 3*x + 1      # a polynomial
frac = (a + b) / (a - b)  # a fraction
eq   = y & x**2 - 5       # an equation (y = x^2 - 5)

```

every expression is a tree. `x**2 + 3*x + 1` is an `Add` node with children `Pow(x, 2)`, `Mul(3, x)`, and `Integer(1)`. you can inspect any expression's children:

```

expr = x**2 + 3*x
expr.children      # [Pow(x, 2), Mul(3, x)]
expr.children[0]   # Pow(x, 2)
expr.children[1]   # Mul(3, x)

```

4.2. variables

pre-defined variables (single-letter, glyph length 1):

```
a, b, c, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z
```

some letters are reserved: `d` (differential), `e` (euler's number), `f`, `g`, `h` (functions), `i` (imaginary unit).

greek letters: `alpha`, `beta`, `gamma`, `theta`, `phi`

special: `dots` (renders `...`), `cdots` (centered dots)

create custom variables:

```

xi = Variable('\\xi', 1)      # symbol, glyph_length
mu = Variable('\\mu', 1)

```

batch-create variables with `Variables`:

```

a, b, c = Variables('abc')
mu, chi = Variables('\\mu', '\\chi')

```

4.3. numbers

class	python type	example
<code>Integer</code>	<code>int</code>	<code>Integer(5)</code> , or just use <code>5</code> in expressions
<code>Real</code>	<code>float</code>	<code>Real(3.14)</code> , <code>e</code> , <code>pi</code> , <code>tau</code>
<code>Rational</code>	<code>Fraction</code>	<code>Rational(1, 3)</code>
<code>Complex</code>	<code>complex</code>	<code>i</code> (the imaginary unit)

you rarely need to construct these directly. when you write `x + 5`, the `5` is automatically converted to `Integer(5)` by the `Smarten` function.

pre-defined number constants:

```

zero, one, two, three, four, five, six, seven, eight, nine, ten
e      (euler's number)
pi     (3.14159 ...)
tau    (2 * pi)
i      (imaginary unit)

```

4.4. operations

all standard operations are created via python operators:

python	expression class	latex	notes
<code>a + b</code>	<code>Add(a, b)</code>	<code>a + b</code>	n-ary (can have 2+ children)
<code>a - b</code>	<code>Sub(a, b)</code>	<code>a - b</code>	auto-parenthesizes right child if needed
<code>a * b</code>	<code>Mul(a, b)</code>	<code>a \cdot b</code> or <code>ab</code>	mode: <code>'dot'</code> , <code>'x'</code> , <code>'juxtapose'</code> , or <code>'auto'</code>
<code>a / b</code>	<code>Div(a, b)</code>	<code>\frac{a}{b}</code>	mode: <code>'fraction'</code> or <code>'inline'</code>
<code>a ** b</code>	<code>Pow(a, b)</code>	<code>a^b</code>	
<code>-a</code>	<code>Negative(a)</code>	<code>-a</code>	

unary operations: `Negative`, `PlusMinus (\pm)`, `MinusPlus (\mp)`

multiplication mode can be controlled globally through `algebra_config['multiplication_mode']` or per-instance:

```

Mul(3, x, mode='dot')      # 3 · x
Mul(3, x, mode='juxtapose') # 3x
Mul(3, x, mode='x')        # 3 × x
Mul(3, x, mode='auto')     # auto: dot for number*number, juxtapose
                           # otherwise

```

division mode:

```

Div(a, b, mode='fraction') # vertical fraction (default)
Div(a, b, mode='inline')   # a ÷ b

```

4.4.1. nesting and de-nesting

expressions with more than 2 children can be restructured:

```

Add(a, b, c).nest('right') # Add(a, Add(b, c))
Add(a, b, c).nest('left')  # Add(Add(a, b), c)
Add(a, Add(b, c)).denest()  # Add(a, b, c)

```

4.4.2. computing values

any expression without variables can be numerically evaluated:

```

expr = (3 + 4) * 2
expr.compute()      # 14
expr.evaluate()     # Integer(14) (returns an Expression)

```

4.5. relations and equations

equations are created with the `&` or `|` operators between expressions:

```

eq = y & x**2 + 1      # y = x^2 + 1
eq = y | x**2 + 1      # same thing

```

note: `|` is also used for equations. this means `|` does not work as bitwise or on expressions; it creates an `Equation`.

other relations:

python	class	latex
<code>a & b</code> or <code>a b</code>	<code>Equation</code>	$a = b$
<code>a < b</code>	<code>LessThan</code>	$a < b$
<code>a > b</code>	<code>GreaterThan</code>	$a > b$
<code>a ≤ b</code>	<code>LessThanOrEqualTo</code>	$a \leq b$
<code>a ≥ b</code>	<code>GreaterThanOrEqualTo</code>	$a \geq b$

access the two sides of a relation:

```

eq = y & x**2
eq.left      # y
eq.right     # x**2

```

4.6. functions

there are three pre-defined single-letter functions: `f`, `g`, `h`.

call a function on an argument:

```

f(x)          # renders as f(x)
f(x + 1)      # renders as f(x + 1)
g(x, y)       # renders as g(x, y) (multi-argument)

```

create custom functions:

```

# with a python rule for computation
F = Function('F', symbol_glyph_length=1, python_rule=lambda x: x**2 + 1)

# with an algebraic rule (an expression that defines the function)

```

```
F = Function('F', symbol_glyph_length=1, algebra_rule=x**2 + 1,
algebra_rule_variables=[x])
```

function composition:

```
(f @ g)(x)    # renders as (f ∘ g)(x)
```

4.6.1. parenthesization modes

when a function is applied, the argument's parentheses are controlled by the function's `parentheses_mode`:

- 'always' (default) - argument always gets parentheses: `f(x)`, `f(x+1)`
- 'strong' - parens only if arg is an operation or negative: `\sin x`, `\sin(x+1)`
- 'weak' - parens only if arg is add/sub or negative: `\sin x`, `\sin x^2`, `\sin(x+1)`
- 'var only' - parens unless arg is a single variable
- 'never' - never adds parens: `|x|`, `\sqrt{x}`

4.6.2. built-in function types

logarithms:

```
Log(base)(x)    # log_base(x)
ln(x)           # natural log
log(x)          # common log (base 10)
```

radicals:

```
sqrt(x)         # square root
cbert(x)        # Rad(3)(x), cube root
Rad(n)(x)       # nth root
```

miscellaneous:

```
abs_val(x)      # |x|
fact(x)         # x!
binom(n, k)     # binomial coefficient (n choose k)
nCk(n, k)       # same, nCk notation
```

big operators:

```
Sum(k, 1, n)(expr)    # summation from k=1 to n
Product(k, 1, n)(expr) # product from k=1 to n
```

big operators can be expanded:

```
s = Sum(k, 1, 5)
s.expand_on_args(k**2)    # 1 + 4 + 9 + 16 + 25
```

4.7. scripts (subscripts and superscripts)

subscripts and superscripts:

```
Subscript(x, 1)    # x_1
Superscript(x, 2)  # x^2 (different from Pow: no mathematical meaning,
just visual)
```

pre-defined subscripted variables: `x1, x2, x3, y1, y2, y3, z1, z2, z3`

batch-create subscripted variables:

```
x1, x2, x3 = Subscripts_L(x, 1, 2, 3)    # shared left: x_1, x_2, x_3
a_x, b_x = Subscripts_R(a, b, x)         # shared right: a_x, b_x

# the same works for superscripts:
y_sq, y_cb = Superscripts_L(y, 2, 3)     # shared left: y^2, y^3
a_sq, b_sq = Superscripts_R(a, b, 2)     # shared right: a^2, b^2
```

4.8. sequences and coordinates

```
Sequence(a, b, c)    # a, b, c
Coordinate(1, 2)     # (1, 2) with auto-parentheses
```

4.9. parentheses

parentheses are managed automatically by default. you can also control them manually:

```
expr = a + b
expr.give_parentheses()    # wraps in ( )
expr.give_parentheses(True, symbols=['[', ']']) # wraps in [ ]
expr.clear_all_parentheses() # removes all parens from the tree
expr.reset_parentheses()   # clears then re-applies auto logic
```

auto-parenthesization follows standard math conventions. for example, `a * (b + c)` auto-wraps the addition in parens. `(a + b)**2` wraps the base.

5. the address system

the address system is fundamental to `mf_algebra`. it provides a way to refer to any subexpression in the tree, and is used heavily for animations.

5.1. what addresses are

an address is a string of digits, where each digit selects a child by index at that depth in the tree.

```
expression: (x + 3) * y
tree:
  Mul
  ├── 0: Add
  │   ├── 0: x      address "00"
  │   └── 1: 3      address "01"
  └── 1: y          address "1"
```

"" → the whole expression: $(x + 3) * y$

"0" → the first child: $x + 3$

"1" → the second child: y

"00" → child 0 of child 0: x

"01" → child 1 of child 0: 3

getting subexpressions by address:

```
expr = (x + 3) * y
expr.get_subex('')      # Mul(Add(x, 3), y) (the whole thing)
expr.get_subex('0')     # Add(x, 3)
expr.get_subex('00')    # x
expr.get_subex('01')    # Integer(3)
expr.get_subex('1')     # y
```

5.2. special address characters

beyond digits, addresses support special characters for targeting specific parts of the rendered latex:

character	meaning
(	left parenthesis glyphs
)	right parenthesis glyphs
_	all glyphs of this expression, excluding any parentheses
!	all glyphs except those at the remainder of the address
+, -, *, /, ^, =, <, >, ,	operator/symbol glyphs (for combinators)
f	the “function name” part of a function application (excludes args and children)
F	all glyphs belonging to the function (including children but excluding args)
c	all child glyphs of a function
~	the unary operator glyph

examples:

```
eq = x + 3 & y
eq.get_glyphs_at_address('=')    # glyph indices for the "=" sign
eq.get_glyphs_at_address('0+')  # glyph indices for the "+" sign on the
left side
eq.get_glyphs_at_address('!1')   # all glyphs except child 1 (y)
```

5.3. glyphs and the mob

every expression produces a tex mobject via `.mob`. the mob is a sequence of glyphs (individual rendered characters). the address system maps addresses to glyph indices, which lets you target specific parts of the rendered output for animation.

```
expr = x**2 + 1
expr.mob                # the Tex mobject
expr.glyph_count       # total number of glyphs
expr[x]                # VGroup of glyphs corresponding to x
expr['0']               # VGroup of glyphs at address '0' (x^2)
expr['01']              # VGroup of glyphs at address '01' (the 2 in x^2)
```

you can also index by expression object:

```
expr[x]                # all glyphs where x appears
```

5.4. querying addresses

```
expr.get_all_addresses()    # list of all valid
addresses
expr.get_all_leaf_addresses() # addresses of childless
nodes
expr.get_all_twig_addresses() # addresses one level above
leaves
expr.get_all_addresses_of_type(Add) # addresses of all Add nodes
expr.get_addresses_of_subex(x)    # addresses where x appears
expr.get_all_addresses_with_paren() # addresses of parenthesized
subexpressions
expr.get_addressbook()          # dict mapping every address
to its glyph indices
```

6. actions

an action transforms an expression into another expression and (optionally) produces an animation for the transition. every action has two core methods:

- `get_output_expression(input_expression)` - returns the transformed expression

- `get_addressmap(input_expression)` - returns a list of address mappings that drive the animation

6.1. built-in actions

6.1.1. substitution

replace subexpressions with other expressions:

```
sub = substitute_({x: 5})

# apply it to get the result
result = sub.get_output_expression(x**2 + 3*x) # 5^2 + 3*5 =
Integer(25) after evaluation

# or use it in a timeline (see below)
```

`substitute_` supports three animation modes:

- `'transform'` (default) - glyphs morph from old to new
- `'swirl'` - glyphs follow an arc path
- `'fade'` - old glyphs fade out, new glyphs fade in

```
substitute_({x: a + b}, mode='swirl', arc_size=3.14)
substitute_({x: a + b}, mode='fade')
```

substitute into an outer expression:

```
sub_into = substitute_into(x**2 + 1)
# substitutes the input expression in place of x in x^2 + 1
result = sub_into.get_output_expression(y + 3) # (y+3)^2 + 1
```

6.1.2. function expansions

expand big operators (like `Sum`) or un-evaluated functions:

```
expand_to_terms()      # expands a big operator: Sum(i, 1, 3)(i^2) →
1^2 + 2^2 + 3^2
apply_func_rule()      # expands a function application according to its
algebra_rule
```

6.1.3. applying operations to both sides

these actions wrap an expression in an operation:

```
add_(5)      # expr → expr + 5
sub_(3)      # expr → expr - 3
mul_(2)      # expr → expr * 2
```

```
div_(4)          # expr → expr / 4
pow_(2)          # expr → expr ^ 2
equals_(0)       # expr → expr = 0
apply_func_(f)   # expr → f(expr)
```

by default, the new operand goes on the right. use `side='left'` to put it on the left:

```
add_(5, side='left') # expr → 5 + expr
```

6.1.4. evaluation

numerically evaluate the “lowest” computable branch of the expression:

```
evaluate_()          # evaluates one twig at a time
evaluate_(allowed_type=Integer) # only evaluate if the result is an Integer
```

6.1.5. permutations and swapping

```
swap_children_()      # swap left and right children
swap = swap_children_() # pre-built instance: swap

permute_children_(Permutation(2, 0, 1)) # reorder children by permutation
```

`Permutation` supports cycle notation:

```
p = Permutation.from_cycles([0, 2], [1, 3])
```

6.1.6. parallel actions

combine multiple actions to run simultaneously:

```
action = add_(5).left() | sub_(3).right() # adds 5 to the left,
subtracts 3 from the right
action = add_(5) | sub_(3)                # parallel on the whole
expression

# or equivalently:
action = ParallelAction(add_(5).left(), sub_(3).right())
```

6.1.7. algebraic actions and template matching

`AlgebraicAction` is a powerful tool for defining transformations via template patterns. you provide two expression templates (input and output), and the action uses `match_expressions` to figure out what corresponds to what.

```
# define an action: a*b + a*c → a*(b+c) (factoring out a)
factor_out = AlgebraicAction(
    a*b + a*c,      # template1 (input pattern)
    a * (b + c),    # template2 (output pattern)
)

# apply it
result = factor_out.get_output_expression(x*y + x*z)  # x*(y+z)
```

the template matching works recursively: it walks both the template and the input expression tree, and builds a dictionary mapping template variables to the actual subexpressions.

you can add conditions to restrict which subexpressions can match a template variable:

```
from MF_Algebra.expressions.numbers.number import Number

rule = AlgebraicAction(
    a + b,
    b + a,
    var_condition_dict={a: lambda exp: isinstance(exp, Number)}
)
```

`AlgebraicAction` also supports `.reverse()` to swap the templates (apply the rule in the other direction).

6.1.8. simplification rules

the library includes 35 pre-defined simplification rules, all subclasses of `SimplificationRule` (which is itself an `AlgebraicAction`):

identity rules:

<code>add_zero_R</code>	$a + 0 \rightarrow a$	<code>add_zero_L</code>	$0 + a \rightarrow a$
<code>sub_zero_R</code>	$a - 0 \rightarrow a$	<code>sub_zero_L</code>	$0 - a \rightarrow -a$
<code>mul_zero_R</code>	$a * 0 \rightarrow 0$	<code>mul_zero_L</code>	$0 * a \rightarrow 0$
<code>mul_one_R</code>	$a * 1 \rightarrow a$	<code>mul_one_L</code>	$1 * a \rightarrow a$
<code>div_one_R</code>	$a / 1 \rightarrow a$	<code>div_zero_L</code>	$0 / a \rightarrow 0$
<code>pow_one_R</code>	$a ^ 1 \rightarrow a$	<code>pow_one_L</code>	$1 ^ a \rightarrow 1$
<code>pow_zero_R</code>	$a ^ 0 \rightarrow 1$	<code>pow_zero_L</code>	$0 ^ a \rightarrow 0$
<code>sub_same</code>	$a - a \rightarrow 0$	<code>div_same</code>	$a / a \rightarrow 1$

inverse rules:

<code>add_sub</code>	$a + b - b \rightarrow a$	<code>sub_add</code>	$a - b + b \rightarrow a$
<code>mul_div</code>	$a * b / b \rightarrow a$	<code>div_mul</code>	$a / b * b \rightarrow a$
<code>pow_root</code>	$\text{nth_root}(a^n) \rightarrow a$	<code>root_pow</code>	$\text{nth_root}(a)^n \rightarrow a$
<code>exp_log</code>	$\log_b(b^a) \rightarrow a$	<code>log_exp</code>	$b^{(\log_b(a))} \rightarrow a$

exponent rules:

pow_mul	$x^a * x^b \rightarrow x^{(a+b)}$
pow_div	$x^a / x^b \rightarrow x^{(a-b)}$
mul_pow	$(a*b)^n \rightarrow a^n * b^n$
div_pow	$(a/b)^n \rightarrow a^n / b^n$
pow_pow	$(x^a)^b \rightarrow x^{(a*b)}$
neg_neg	$-(-a) \rightarrow a$
neg_pow_A	$x^{(-n)} \rightarrow 1/x^n$
neg_pow_B	$(a/b)^{(-n)} \rightarrow (b/a)^n$

radical rules:

root_to_power_A	$\text{nth_root}(x) \rightarrow x^{(1/n)}$
root_to_power_B	$\text{nth_root}(x^a) \rightarrow x^{(a/n)}$
root_mul	$\text{nth_root}(a*b) \rightarrow \text{nth_root}(a) * \text{nth_root}(b)$
root_div	$\text{nth_root}(a/b) \rightarrow \text{nth_root}(a) / \text{nth_root}(b)$

log rules:

log_mul	$\log_b(x*y) \rightarrow \log_b(x) + \log_b(y)$
log_div	$\log_b(x/y) \rightarrow \log_b(x) - \log_b(y)$
log_pow	$\log_b(x^a) \rightarrow a * \log_b(x)$
log_change_base	$\log_a(x) \rightarrow \log_b(x) / \log_b(a)$

every rule class whose name ends in `_` automatically creates an instance with the name minus the trailing underscore (e.g., class `add_zero_R_` creates instance `add_zero_R`).

use a simplification rule:

```
result = add_zero_R.get_output_expression(x + 0)    # x
```

6.1.9. equation maneuvers

`EquationManeuver` is a subclass of `AlgebraicAction` that transforms equations. these are used by the `Solve` timeline to automatically solve equations.

built-in maneuvers:

alg_add_R	$a + b = c \rightarrow a = c - b$	(subtract b from both sides)
alg_add_L	$a + b = c \rightarrow b = c - a$	(subtract a from both sides)
alg_mul_R	$a * b = c \rightarrow a = c / b$	(divide by b)
alg_mul_L	$a * b = c \rightarrow b = c / a$	(divide by a)
alg_pow_2_R	$a^2 = b \rightarrow a = \pm\text{sqrt}(b)$	(square root both sides)

alg_pow_R	$a^b = c$	$\rightarrow$	$a = \text{bth_root}(c)$	(nth root both sides)
alg_pow_L	$a^b = c$	$\rightarrow$	$b = \text{log}_a(c)$	(logarithm both sides)
alg_pow_e_L	$e^a = b$	$\rightarrow$	$a = \ln(b)$	(natural log both sides)
alg_neg_R	$-a = b$	$\rightarrow$	$a = -b$	(negate both sides)

each maneuver supports `.reverse()` (undo), `.flip()` (swap sides of the equation), and `.reverse_flip()`.

6.1.10. wrapping and reciprocals

some other miscellaneous algebraic actions are available for manipulating expressions:

<code>wrap_subex_(expr)</code>	<code># x $\rightarrow$ expr(x)</code>
<code>unwrap_subex_()</code>	<code># f(x) $\rightarrow$ x</code>
<code>rewrap_subex_(expr)</code>	<code># f(x) $\rightarrow$ expr(x)</code>
<code>reciprocal_()</code>	<code># a/b $\rightarrow$ b/a or a $\rightarrow$ 1/a</code>

6.1.11. distribution and factoring

<code>distribute_()</code>	<code># auto-detect direction</code>
<code>distribute_(mode='left')</code>	<code># distribute left: $a*(b+c) \rightarrow a*b + a*c$</code>
<code>distribute_(mode='right')</code>	<code># distribute right: $(a+b)*c \rightarrow a*c + b*c$</code>

<code>difference_of_squares_()</code>	<code># $a^2 - b^2 \rightarrow (a+b)(a-b)$</code>
---------------------------------------	--

6.1.12. action modifiers

every action supports modifiers that control which part of the expression it targets:

<code>action.pread('0')</code>	<code># preaddress: apply the action only to child 0</code>
<code>action.pread('1')</code>	<code># apply only to child 1</code>
<code>action.left()</code>	<code># shortcut for .pread('0')</code>
<code>action.right()</code>	<code># shortcut for .pread('1')</code>
<code>action.both()</code>	<code># apply to children 0 and 1 (good for both sides</code>
of an equation)	
<code>action.both(3)</code>	<code># apply to children 0, 1, and 2</code>

calling `.pread()` with multiple addresses creates a `ParallelAction`:

<code>evaluate_().pread('00', '10')</code>	<code># evaluate at two addresses</code>
simultaneously	

6.1.13. address maps and animations

an address map is a list of entries, each of the form `[from_address, to_address, kwargs]`:

- `from_address` - address in the input expression (string, or `[]` for “fade in”)

- `to_address` - address in the output expression (string, or `[]` for “fade out”)
- `kwargs` - optional dict of animation parameters (`path_arc` , `delay` , `run_time` , etc.)

example address map for $a + b = c \rightarrow a = c - b$:

```
addressmap = [
    ['01', '11', {'path_arc': 3}],    # b moves from left side (+b) to
right side (-b)
    ['0+', '1-', {'path_arc': 3}],    # the + sign transforms into a -
sign
]
```

the `TransformByAddressMap` animation converts address maps to glyph maps and creates the actual manim animation. this is wrapped automatically when you use actions.

6.1.13.1. low-level animation variants

for maximum control:

```
# directly specify an address map and expressions
AddressMapAction(['0', '1', {'path_arc': 2}], ['1', '0', {'path_arc':
2}])

# directly specify a glyph map
GlyphMapAction(input_exp, output_exp, ([0,1,2], [3,4,5], {'path_arc':
2}))

# use a raw animation callable
AnimationAction(my_animation_function)
```

6.1.14. custom actions

subclass `Action` and implement:

1. `get_output_expression(self, input_expression)` - return the new expression
2. `get_addressmap(self, input_expression)` - return a list of `[from, to, kwargs]` entries

```
class my_action_(Action):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)

    def get_output_expression(self, input_expression):
        # your transformation logic
        return new_expression

    def get_addressmap(self, input_expression):
        return [
```



```

        ['0', '0'],          # child 0 maps to child 0
        ['1', '1'],          # child 1 maps to child 1
        [[], '2'],           # child 2 fades in
    ]

```

decorators are automatically applied to all `Action` subclasses:

- `preaddressfunc` - makes `get_output_expression` respect `self.preaddress`
- `preaddressmap` - shifts all addresses in the address map by the preaddress
- `autoparenmap` - automatically handles parentheses appearing or disappearing
- `autokwargmap` - applies default introducer/remover kwargs to fade-in/fade-out entries

7. timelines

7.1. building timelines with `>>`

the `>>` operator is the primary way to build timelines. it chains expressions, actions, and other timelines together:

```

# expression >> action → timeline (with output auto-computed)
T = x**2 + 3*x >> substitute_({x: 2})
# T.expressions = [x^2 + 3x, 2^2 + 3*2]

# expression >> action >> action → longer timeline
T = (y & x**2 - 4) >> sub_(x**2).both() >> evaluate_().right()

# expression >> expression → timeline with no action (will use
TransformMatchingTex)
T = (x + 1) >> (x + 2)

```

you can chain as many steps as you want:

```

T = (y & 2*x + 3) >> sub_(3).both() >> div_(2).both() >>
evaluate_().right()
# steps through: y = 2x+3 → y-3 = 2x+3-3 → y-3 = 2x → ...

```

7.2. manual timelines

```

T = Timeline()
T.add_expression_to_start(y & x**2 + 1)
T.add_action_to_end(substitute_({x: 3}))
# the output expression is computed automatically (auto_propagate=True
by default)

```

you can also set steps manually:

```
T.set_expression(0, some_expression)
T.set_action(0, some_action)
```

7.3. auto timelines

`AutoTimeline` automatically fills in actions by calling its `decide_next_action()` method. once you add an expression, it tries to figure out what action to apply, and then computes the next expression, and so on.

subclass it and implement `decide_next_action(self, index)`:

```
class MyTimeline(AutoTimeline):
    def decide_next_action(self, index):
        last_exp = self.get_expression(index)
        # your logic here
        return some_action_or_None
```

you can pause and resume auto-filling:

```
T.suspend()    # stops auto-filling
T.resume()     # resumes auto-filling
```

7.4. evaluate timeline

`Evaluate` is an auto-timeline that repeatedly evaluates the lowest-level computable subexpressions:

```
T = Evaluate(first_expression=(3 + 4) * (2 + 1))
# automatically fills in:
#   (3+4)*(2+1) → 7*(2+1) → 7*3 → 21

# mode controls how many branches are evaluated per step:
T = Evaluate((3 + 4) * (2 + 1), mode='one at a time') # one twig per
step (default)
T = Evaluate((3 + 4) * (2 + 1), mode='all at once')   # all twigs in
one step
```

7.5. solve timeline

`Solve` is an auto-timeline that solves an equation for a given variable:

```
T = Solve(solve_for=x, first_expression=(y & 2*x + 3))
# automatically fills in steps to isolate x:
#   y = 2x + 3 → y - 3 = 2x → (y-3)/2 = x → x = (y-3)/2
```

it does this by trying all registered `EquationManeuver` subclasses and picking the one that brings the solve variable closest to isolation.

access the solution:

```
T.solution # returns the expression on the other side from the solve variable
```

other timeline types:

- `EvaluateAndSolve` - tries evaluate rules first, then solve rules
- `SolveAndEvaluate` - tries solve rules first, then evaluate rules

7.6. playing animations

in a manim scene:

```
class MyScene(Scene):
    def construct(self):
        T = (x**2 + 1) >> substitute_({x: 3}) >> evaluate_()
        self.add(T.mob) # add the current expression's mobject
        T.play_all(self) # play all animations in sequence
```

finer control:

```
T.play_next(self) # play only the next step
T.play_animation(self, index=0) # play a specific step
T.play_range(self, 0, 3) # play steps 0 through 2
T.get_animation(0) # get the animation object without playing it
```

`TimelineScene` is a convenience base class:

```
class MyScene(TimelineScene):
    def construct(self):
        self.timeline >> (x**2 + 1) >> substitute_({x: 3}) >>
evaluate_()
        self.timeline.play_all(self)
```

7.7. timeline options

```
T = Timeline(
    auto_color={x: RED, y: BLUE}, # auto-color subexpressions
    auto_scale=0.8, # scale all mobjects
    auto_fit=[width, height, None], # fit to width/height
    maintain_position=True, # keep expressions centered
    (default True))
```

```

        show_past_steps=True,                # show previous steps
    stacked above
        past_steps_opacity=0.4,             # opacity of past steps
        past_steps_direction=UP,            # direction to shift past
    steps
        past_steps_buff=1,                  # spacing between past steps
        past_steps_shift_run_time=1,         # run time for the shift
    animation
        past_steps_shift_rate_func=smooth    # rate function for the
    shift animation
    )

```

other timeline methods:

```

T.undo_last_action()    # remove the last action/expression
T.reset()               # reset playback to the start
T.align_on_equals()     # align all expressions by their = sign
T.vgroup                # VGroup of all expression mobjects
T.save_to_file('name')  # serialize to saved_objects/name

```

8. calculus

8.1. differentials

```

d(x)    # dx (differential operator)
dx       # pre-defined shortcut (also dy, dz, dr, dtheta, ds, dt, du, dv)

dd(x)   # d/dx (differential quotient operator, renders as a fraction)

prime(f(x))  # f(x)' (prime notation)

```

8.2. differentiation

the `Differentiate` auto-timeline applies derivative rules step by step:

```

T = Differentiate()
T.add_expression_to_start(d(x**3))
# automatically applies PowerRule: d(x^3) → 3*x^2*dx

```

built-in derivative rules:

ConstantRule	$d(n) \rightarrow 0$
ConstantMultipleRule	$d(n*x) \rightarrow n*d(x)$
SumRule	$d(x+y) \rightarrow d(x) + d(y)$
DifferenceRule	$d(x-y) \rightarrow d(x) - d(y)$
ProductRule	$d(x*y) \rightarrow y*d(x) + x*d(y)$
QuotientRule	$d(x/y) \rightarrow (y*d(x) - x*d(y)) / y^2$
PowerRule	$d(x^n) \rightarrow n*x^{(n-1)}*d(x)$
ExponentialRule	$d(n^x) \rightarrow \ln(n)*n^x*d(x)$

function derivatives are also defined for: `e^x`, `ln(x)`, `sin(x)`, `cos(x)`, `tan(x)`, `arcsin(x)`, `arccos(x)`, `arctan(x)`.

8.3. integrals

```
Integral(0, 1)(f(x) * dx)      # definite integral from 0 to 1
I(f(x) * dx)                  # indefinite integral (I is a pre-
defined instance)
PlugInBounds(0, 1, variable=x) # the evaluation bar notation: F(x) |
_0^1
```

like `Differentiate`, the `Integrate` auto-timeline applies integration rules step by step:

```
T = Integrate(first_expression=I(x**2 * dx))
# automatically applies Int_PowerRule_dx_: I(x^2 * dx) → 1/3 * x^3
```

built-in integral rules (subclasses of `IntegralRule`):

```
FundThmCalc_1_      d(Integral(n, x)(u * dx)) -> u
FundThmCalc_2_      Integral(a, b)(du) -> PlugInBounds(a, b, x)
(u)
Int_dx_             I(d(u)) -> u
d_Int_x_            d(I(u)) -> u
Int_ConstantMultiple_ I(n * u) -> n * I(u)
Int_ConstantMultiple_dx_ I(n * u * dx) -> n * I(u * dx)
Int_SumRule_        I(a + v) -> I(a) + I(v)
Int_SumRule_dx_     I((u + v) * dx) -> I(u * dx) + I(v * dx)
Int_DifferenceRule_  I(a - v) -> I(a) - I(v)
Int_DifferenceSumRule_dx_ I((u - v) * dx) -> I(u * dx) - I(v * dx)
Int_PowerRule_dx_   I(x^n * dx) -> 1/(n+1) * x^(n+1)
IntegrationByParts_  I(u * dv) -> u * v - I(v * du)
```

8.4. limits

```
Limit(x, 0)(f(x))      # lim_{x→0} f(x)
Limit(x, inf)(1/x)     # lim_{x→∞} 1/x
```

```
Limit(x, 0, direction='right')(f(x)) #  $\lim_{x \rightarrow 0^+} f(x)$ 
Limit(x, 0, direction='left')(f(x))  #  $\lim_{x \rightarrow 0^-} f(x)$ 
```

`inf` is a pre-defined `Infinity` expression.

8.5. series

infinite series expressions are built around `Sum` and `Series`. a convenience function `Taylor` can generate common taylor series:

```
from MF_Algebra.calculus.series import Series, Taylor

Series(1 / 2**n, variable=n, start=1, end=inf) # generic series

Taylor(sin(x))      # Taylor series for sin(x) centered at 0
Taylor(e**x)        # Taylor series for e^x centered at 0
Taylor(1/(1-x))     # Taylor series for 1/(1-x) centered at 0
```

9. trigonometry

pre-defined trig functions: `sin`, `cos`, `tan`, `csc`, `sec`, `cot`

```
sin(x)              # renders: sin(x) or sin x depending on
parentheses_mode
cos(pi / 4)         # evaluates to sqrt(2)/2 via the unit circle lookup
```

inverse trig (defined in the trigonometry module): `arcsin`, `arccos`, `arctan`, `arccsc`, `arcsec`, `arccot`

trig identities are defined as `TrigIdentity` (subclass of `AlgebraicAction`):

<code>pythagorean</code>	$\sin^2(t) + \cos^2(t) \rightarrow 1$
<code>pythagorean_tan</code>	$\tan^2(t) + 1 \rightarrow \sec^2(t)$
<code>pythagorean_cot</code>	$1 + \cot^2(t) \rightarrow \csc^2(t)$
<code>double_angle_sin</code>	$\sin(2t) \rightarrow 2*\sin(t)*\cos(t)$
<code>double_angle_cos</code>	$\cos(2t) \rightarrow \cos^2(t) - \sin^2(t)$
<code>double_angle_tan</code>	$\tan(2t) \rightarrow 2*\tan(t) / (1 + \tan^2(t))$
<code>sin_add</code>	$\sin(a+b) \rightarrow \sin(a)\cos(b) + \cos(a)\sin(b)$
<code>sin_sub</code>	$\sin(a-b) \rightarrow \sin(a)\cos(b) - \cos(a)\sin(b)$
<code>cos_add</code>	$\cos(a+b) \rightarrow \cos(a)\cos(b) - \sin(a)\sin(b)$
<code>cos_sub</code>	$\cos(a-b) \rightarrow \cos(a)\cos(b) + \sin(a)\sin(b)$

also: `def_tan` ($\tan = \sin/\cos$), `def_cot`, `def_sec`, `def_csc`.

10. sets

```
A = SymbolSet(symbol='A', symbol_glyph_length=1)
S = ElementsSet(1, 2, 3)           # {1, 2, 3}
B = SetBuilder(x, x > 0)           # {x | x > 0}
```

pre-defined number sets: `ZZ` (integers), `NN` (naturals), `RR` (reals), `Empty` (empty set)

set operations via python operators:

python	operation	latex
<code>A & B</code>	intersection	$A \cap B$
<code>A B</code>	union	$A \cup B$
<code>A - B</code>	difference	$A \setminus B$
<code>A * B</code>	cartesian product	$A \times B$
<code>A ** n</code>	cartesian power	A^n
<code>x % A</code>	element of	$x \in A$
<code>A < B</code>	proper subset	$A \subset B$
<code>A ≤ B</code>	subset	$A \subseteq B$

intervals:

```
from MF_Algebra.sets.intervals import OpenInterval, ClosedInterval,
HalfOpenInterval
OpenInterval(0, 1)      # (0, 1)
ClosedInterval(0, 1)    # [0, 1]
```

membership testing:

```
3 in ZZ    # True
3 in NN    # True
```

11. logic

logical operations:

```
Not(P)      # ¬P
And(P, Q)   # P ∧ Q
Or(P, Q)    # P ∨ Q
Xor(P, Q)   # P ⊕ Q
Implies(P, Q) # P → Q
Iff(P, Q)   # P ↔ Q
```

quantifiers:

```
ForAll(x)(P(x))      #  $\forall x P(x)$ 
Exists(x)(P(x))      #  $\exists x P(x)$ 
UniqueExists(x)(P(x)) #  $\exists! x P(x)$ 
ForAll(x, set=RR)(P(x)) #  $\forall x \in \mathbb{R} P(x)$ 
```

12. utilities

12.1. configuration

the global `algebra_config` dictionary controls default behavior:

```
from MF_Algebra import algebra_config

algebra_config['auto_parentheses'] # True - auto-add parens (default
True)
algebra_config['multiplication_mode'] # 'auto' - how to render
multiplication
algebra_config['division_mode']      # 'fraction' - how to render
division
algebra_config['decimal_precision']  # 4 - decimal places for Real
display
algebra_config['integer_tolerance']  # 1e-9 - closeness threshold for
float→int
algebra_config['always_color']       # {} - dict of subexpression-
>color to always apply
algebra_config['default_color']      # '#FFFFFF' - default text color
algebra_config['fast_paren_length']  # True - assume parens are 1
glyph (faster)
algebra_config['fast_glyph_count']   # True - use formula-based glyph
counting (faster)
algebra_config['fast_root_length']   # True - assume radical is 2
glyphs (faster)
```

12.2. smarten

`Smarten` is the type-coercion function used throughout the library. it converts python primitives into expression objects:

input	output
<code>int</code>	<code>Integer</code>
<code>float</code> (close to int)	<code>Integer</code>
<code>float</code>	<code>Real</code>

<code>complex</code>	<code>Complex</code>
<code>Fraction</code>	<code>Rational</code>
<code>Decimal</code>	<code>Real</code> (or <code>Integer</code>)
<code>tuple</code>	<code>Coordinate</code>
<code>...</code> (ellipsis)	<code>dots</code>
<code>np.inf</code>	<code>inf</code>
any <code>MF_Base</code> subclass	returns a copy

12.3. conversions

```
# mf_algebra expression → sympy expression
from MF_Algebra.utils.conversions import MF_Algebra_to_sympy
sympy_expr = MF_Algebra_to_sympy(x**2 + 1)

# text string → mf_algebra expression
from MF_Algebra.utils.conversions import text_to_MF_Algebra
expr = text_to_MF_Algebra("3x^2 + 2x + 1")
```

`text_to_MF_Algebra` handles implicit multiplication (`3x` -> `3*x`), caret for power (`^` -> `**`), and `=` for equations.

12.4. serialization

save and load expressions or timelines using dill:

```
from MF_Algebra.utils.serialization import save_to_file, load_from_file

save_to_file(my_timeline, 'my_solve')    # saves to saved_objects/
my_solve
loaded = load_from_file('my_solve')      # loads it back
```

timelines also have a convenience method:

```
T.save_to_file('my_solve')
```

12.5. random expression generation

```
from MF_Algebra.utils.generation import random_number_expression

expr = random_number_expression(
    leaves=range(-5, 10),      # possible leaf values
    max_depth=3,               # maximum tree depth
    max_children_per_node=2     # max children for Add/Mul
)
```

13. operator overload reference

this is a complete reference of all python operators overloaded by the library.

13.1. expression operators

operator	result	notes
<code>a + b</code>	<code>Add(a, b)</code>	
<code>a - b</code>	<code>Sub(a, b)</code>	
<code>a * b</code>	<code>Mul(a, b)</code>	
<code>a / b</code>	<code>Div(a, b)</code>	
<code>a ** b</code>	<code>Pow(a, b)</code>	
<code>-a</code>	<code>Negative(a)</code>	
<code>a & b</code>	<code>Equation(a, b)</code>	
<code>a b</code>	<code>Equation(a, b)</code>	same as <code>&</code> for expressions
<code>a < b</code>	<code>LessThan(a, b)</code>	
<code>a > b</code>	<code>GreaterThan(a, b)</code>	
<code>a ≤ b</code>	<code>LessThanOrEqualTo(a, b)</code>	
<code>a ≥ b</code>	<code>GreaterThanOrEqualTo(a, b)</code>	
<code>a @ dict</code>	<code>a.substitute(dict)</code>	substitution
<code>a @ b</code>	<code>Composition(a, b)</code>	function composition (both must be functions)
<code>a(x)</code>	<code>ApplyFunction(a, x)</code>	function application (a must be a function)
<code>a >> b</code>	<code>combine_to_timeline(a, b)</code>	timeline construction
<code>a[key]</code>	glyphs at address/subex	mob glyph access
<code>len(a)</code>	<code>a.glyph_count</code>	

13.2. action operators

operator	result	notes
<code>a b</code>	<code>ParallelAction(a, b)</code>	combine actions
<code>a + b</code>	<code>ParallelAction(a, b)</code>	same as <code> </code>
<code>expr ≥ action</code>	<code>action.get_output_expression(expr)</code>	apply action to expression
<code>a >> b</code>	<code>combine_to_timeline(a, b)</code>	timeline construction

13.3. the `>>` pipeline

the `>>` operator is defined on `MF_Base` (the parent of `Expression`, `Action`, and `Timeline`). it delegates to `combine_to_timeline`, which handles all combinations:

left <code>>></code> right	result
expression <code>>></code> expression	timeline with both expressions

expression >> action	timeline with expression, action, and auto-computed output
action >> expression	timeline with action and expression
expression >> timeline	expression prepended to timeline
timeline >> expression	expression appended to timeline
timeline >> action	action appended to timeline
timeline >> timeline	timelines concatenated

14. project structure

```

src/MF_Algebra/
  __init__.py
  expressions/
    expression_core.py      # Expression base class, Address,
ExpressionContainer
  variables.py              # Variable, pre-defined vars (a-z, greek),
Variables
  numbers/
    number.py               # Number base class
    integer.py              # Integer, zero-ten constants
    real.py                 # Real, e, pi, tau
    rational.py             # Rational
    complex.py              # Complex, i
  combiners/
    combiners.py            # Combiner base class, CombinerContainer
    operations.py           # Add, Sub, Mul, Div, Pow, Negative,
PlusMinus, etc.
    relations.py            # Equation, LessThan, GreaterThan, etc.
    scripts.py              # Subscript, Superscript, Subscripts_L/R,
etc.
    sequences.py            # Sequence, Coordinate
  functions/
    functions.py            # Function, ApplyFunction, Composition, f/
g/h
    logarithms.py           # Log, NaturalLog (ln), CommonLog (log)
    radicals.py             # Rad, SquareRoot (sqrt), cbrt
    miscellaneous.py        # AbsoluteValue, Factorial,
BinomialCoefficient
    operators.py            # BigOperator, Sum, Product
  actions/
    action_core.py          # Action base class, decorators,
IncompatibleExpression
  animations.py             # TransformByAddressMap

```

substitution.py	# substitute_, substitute_into_
apply_operation.py	# add_, sub_, mul_, div_, pow_, equals_,
apply_func_	
evaluation.py	# evaluate_
permutations.py	# Permutation, permute_children_,
swap_children_	
parallel.py	# ParallelAction
variants.py	# AddressMapAction, GlyphMapAction,
AnimationAction	
expand_functions.py	# expand_to_terms, apply_func_rule
timelines/	
timeline_core.py	# Timeline, TimelineScene
timeline_variants.py	# AutoTimeline, CombinedRuleTimeline
timeline_common.py	# Evaluate, Solve, EvaluateAndSolve,
SolveAndEvaluate	
algebra/	
algebra_core.py	# AlgebraicAction, match_expressions
equations.py	# EquationManeuver and all alg_* maneuvers
simplify.py	# SimplificationRule and ~35 rules
distribution.py	# distribute_
factoring.py	# difference_of_squares_
reciprocal.py	# reciprocal_
wrap.py	# rewrap_subex_, wrap_subex_, unwrap_subex_
calculus/	
differentials.py	# d, dx, dy, ..., dd(), prime
differentiation.py	# DerivativeRule subclasses, Differentiate
timeline	
integrals.py	# Integral, IndefiniteIntegral,
PlugInBounds	
limits.py	# Limit, Infinity (inf), Approaches
series.py	
integration.py	
trigonometry/	
trigonometry_core.py	# sin, cos, tan, csc, sec, cot
identities.py	# TrigIdentity subclasses (pythagorean,
double angle, etc.)	
inverses.py	# arcsin, arccos, arctan, etc.
powers.py	
unit_circle.py	# unit circle lookup table
sets/	
set_core.py	# Set, ElementsSet, SymbolSet, SetBuilder,
ZZ, NN, RR, Empty	
operations.py	# Union, Intersection, Difference,
CartesianProduct, etc.	
intervals.py	# OpenInterval, ClosedInterval,
HalfOpenInterval	

```
logic/
  quantifiers.py      # ForAll, Exists, UniqueExists
  operations.py       # Not, And, Or, Xor, Implies, Iff
  booleans.py
utils/
  config.py           # algebra_config dictionary
  type_fixing.py      # Smarten function
  base_class.py       # MF_Base, combine_to_timeline
  addresses.py        # apply_addressmap utility
  conversions.py      # MF_Algebra_to_sympy, text_to_MF_Algebra
  serialization.py    # save_to_file, load_from_file
  generation.py       # random_number_expression
  tex_strings.py      # add_spaces_around_brackets
  debugging.py        # debugging utilities
```